CYBER SECURITY / INTERNSHIP TASK

API Security Risk Analysis

Read-only observations of the JSONPlaceholder demo API

Prepared by Sumit Rathod	**Assessment date** 23 October 2026
Assessment type Read-only / educational review	**Deliverable** Internship report

ASSESSMENT SNAPSHOT

Two documented GET requests returned expected synthetic demo data. No exploitable vulnerability was confirmed within this narrow test scope.

Evidence-based observations • practical recommendations • clearly stated limitations

Executive summary

I sent two ordinary GET requests to documented JSONPlaceholder demo endpoints /posts/1 and /users/1. Both returned HTTP 200 and synthetic demonstration records. Captured responses included X-Content-Type-Options: nosniff, rate-limit headers, and X-Powered-By: Express. The user endpoint includes contact-shaped fields in fake data; this should not be interpreted as exposure of real personal data.

This exercise did not test authorization boundaries, BOLA/IDOR, authentication, input validation, write operations, or CORS behavior. No attempt was made to enumerate records, bypass controls, fuzz inputs, or load-test the service. The findings below are configuration and design observations, not confirmed vulnerabilities.

Scope and rules

- **In scope:** public documentation and two documented JSONPlaceholder GET endpoints.
- **Requests:** GET /posts/1; GET /users/1.
- **Permitted method used:** passive/read-only HTTP observation; one request per endpoint.
- **Not performed:** POST/PUT/PATCH/DELETE; login or token testing; ID enumeration; authorization bypass; fuzzing; brute force; scanning; rate-limit stress; denial-of-service; tests against private or production APIs.
- **Evidence handling:** response headers were recorded and contact-like values in the user object were omitted from the report.

Method and observed results

Requests were made with PowerShell Invoke-WebRequest on 23 October 2026. The two GET request definitions are included in evidence/requests.http and evidence/postman_collection.json; sanitized response observations are recorded in evidence/api-get-evidence.txt.

Request	Result	Relevant observations
GET https://jsonplaceholder.typicode.com/posts/1	200	JSON response with a single fake post record; X-Content-Type-Options: nosniff; X-Powered-By: Express; rate-limit headers present.
GET https://jsonplaceholder.typicode.com/users/1	200	JSON response with synthetic user fields including contact-shaped keys; values omitted; same selected headers observed.

Findings

API-01 — Anonymous reads return demo records

Severity: Low / Informational — expected demo behavior **Evidence:** Both documented GET requests returned HTTP 200 without credentials. JSONPlaceholder describes itself as a free fake online REST API for testing and prototyping.

Impact: This observation is not a vulnerability for the documented demo service. For a production API, public access to user or business records could expose data if those records are not intended to be public.

Recommendation: Production API owners should define access rules per endpoint and enforce authentication and authorization server-side wherever data is private. This finding does not establish that the demo API has broken access control.

API-02 — Demo user object contains contact-shaped fields

Severity: Low / Informational — synthetic data; no real personal data confirmed **Evidence:** The /users/1 response contained fields such as email, phone, website, address, and company. Values are not reproduced. JSONPlaceholder identifies its content as fake data.

Impact: A field's presence alone does not prove sensitive data exposure. The same schema on a production API could disclose more information than a client needs.

Recommendation: For production APIs, return only fields required for the caller's use case, classify data before publication, and use synthetic values in demos and development environments.

API-03 — Express framework fingerprint disclosed

Severity: Low **Evidence:** Both observed responses included X-Powered-By: Express.

Impact: This header reveals a technology detail that may help an attacker choose research topics. It does not prove a vulnerable framework version or exploitable weakness.

Recommendation: Production owners may suppress unnecessary technology-identifying headers at the application or edge layer and should keep dependencies patched. No version-specific claim was made from this header.

Control checklist

Control area	Assessment status	Result / limitation
Authentication	Not assessed	Public demo reads are expected; no login/token flow tested.
Authorization / BOLA	Not assessed	No object-ID substitution or cross-user access test was performed.
Excessive data exposure	Limited observation	Demo user schema has contact-shaped fields; values are synthetic. No production data risk established.
Rate limiting	Observed, not stress-tested	x-ratelimit-limit and x-ratelimit-remaining appeared in both responses. No conclusion about enforcement under load.
Response hardening	Partially observed	X-Content-Type-Options: nosniff present; X-Powered-By: Express present.
CORS	Not assessed	No Origin header was sent; no CORS misconfiguration can be concluded.
Input validation	Not assessed	No parameters or payloads were submitted.

Remediation priorities for a production API owner

- Map every endpoint and data field to an intended audience; require authentication and server-side authorization for private resources.
- Test object-level authorization using accounts and records explicitly provisioned for an authorized test, with written scope.
- Minimize response fields and prevent real personal or confidential data from entering demo fixtures.
- Define rate limits based on risk and verify them using an approved, bounded test plan; do not infer protection from a header alone.
- Remove unneeded framework-identification headers and maintain dependency updates.
- Test CORS, validation, and token handling only with explicit authorization and a documented test environment.

Limitations

This was a small, unauthenticated observation of a public fake-data API. It cannot establish the security posture of a production system. The absence of a finding in an untested control area is not evidence that the control is secure. No writes, destructive requests, exploitation, or load testing were performed. Evidence consists of sanitized response notes and repeatable GET request definitions.

Sources

- Future Interns Task 3: https://futureinterns.com/cyber-security-task-3-2026/
- JSONPlaceholder documentation: https://jsonplaceholder.typicode.com/ · Observed post: https://jsonplaceholder.typicode.com/posts/1 · Observed user: https://jsonplaceholder.typicode.com/users/1
- OWASP API Security: https://github.com/OWASP/API-Security · Shieldfy API Security Checklist: https://github.com/shieldfy/API-Security-Checklist